MSML606 HW3 – Analysis Report

Hash Table Implementation and Performance Analysis

AI USAGE REPORT - AI tools (ChatGPT, Claude) were used for general understanding and clarification of concepts (trees, stacks).

1. Implementation Details

For this homework, I implemented a hash table from scratch in Python without using Python's built-in dictionary functions or any other built-in hashing libraries/modules. The hash table stores key-value pairs using a Python list as the underlying array structure.

Each position in the array contains a small list, which stores multiple elements when collisions occur. This technique is called chaining. If multiple keys map to the same index, they are stored together in the list at that index.

Hash Functions - Before computing the hash value, keys must first be converted into an integer. I implemented a simple function that converts keys to numbers:

- If the key is an integer, the absolute value is used.
- If the key is a string, the ASCII values of its characters are summed and then used.
- For other types, the key is converted to a string and processed similarly.

Two hashing methods were implemented:

1. Division Method - The division method calculates the index using the formula:

index = key_number % table_size

This method is simple and fast.

2. Multiplication Method - The multiplication method computes the hash value using a constant between 0 and 1:

index = floor(table_size × ((key_number × A) mod 1))

here A is a constant (0.618). This method often distributes keys more evenly than the division method.

Collision Resolution - I used chaining to resolve collisions. In this method, each array slot stores a list (chain) of key–value pairs.

When inserting a key:

1. First the hash index is computed.
2. The chain at that index is checked.
3. If the key already exists, its value is updated.
4. Otherwise, the new pair is appended to the chain.

The main operations are implemented as follows:

insert(key, value) - Computes the hash index and appends the key–value pair to the chain if the key does not already exist.

search(key) - Computes the hash index and scans the chain at that location to find the key.

delete(key) - Finds the key in the chain and removes it from the list.

Dynamic Resizing - Dynamic resizing was not implemented in this solution because chaining can still function correctly even when the load factor exceeds 1. However, without resizing, performance may degrade when many elements are inserted, since chains become longer and searches take more time.

2. Performance Comparison

To evaluate the performance of the hash table, several experiments were conducted using different table sizes, load factors, and key distributions.

Search time was measured using time.perf_counter(), which provides high-precision timing in Python. For each test case, the average search time was calculated over multiple searches.

Both successful searches (searching for keys that exist in the table) and unsuccessful searches (searching for keys that are not present) were measured.

Load Factor vs Search Time

The load factor is defined as:

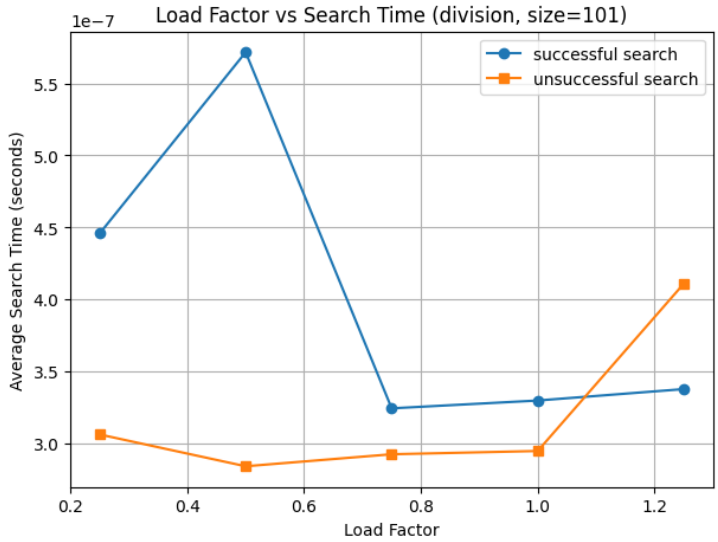
load factor = number of elements / table size

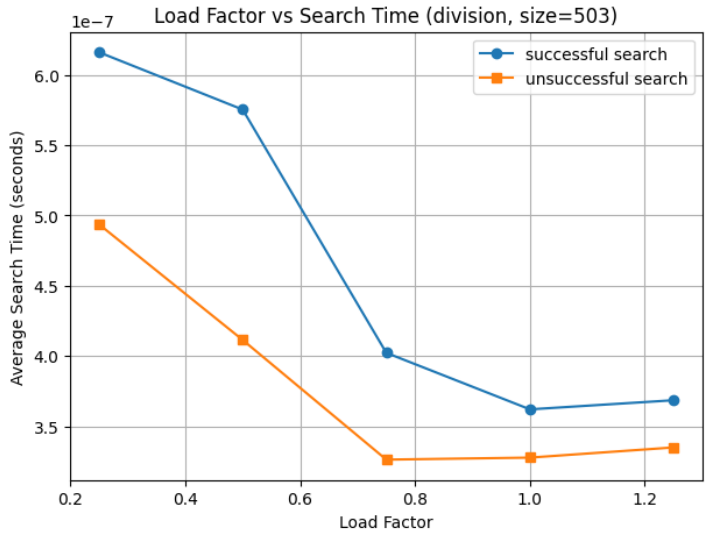
Experiments were conducted using multiple table sizes (for example 101, 503, and 1009) and different load factors.

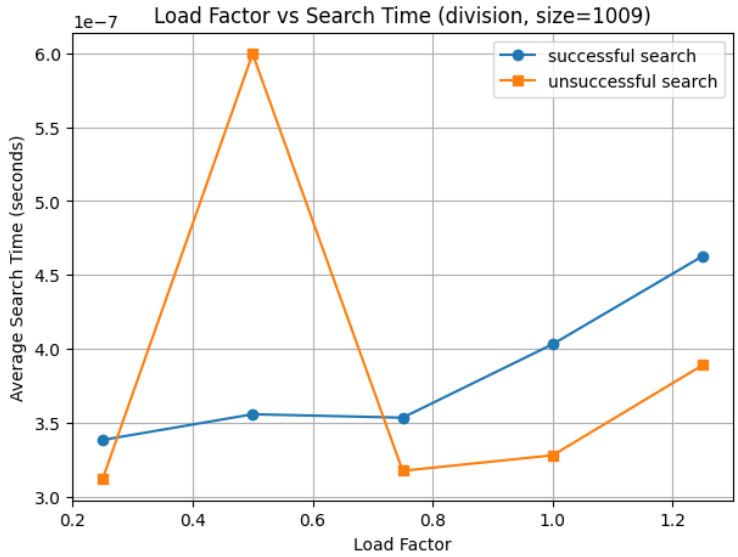
In general, the results showed that

- When the load factor is small, search time is very fast because chains remain short.
- As the load factor increases, more collisions occur.
- This leads to longer chains and slower search operations.

Unsuccessful searches often take slightly longer than successful searches because the entire chain may need to be scanned before determining that the key does not exist.


Load Factor vs Search Time (division, size=101)
1e−7
successful search
unsuccessful search
Average Search Time (seconds)
Load Factor


Load Factor vs Search Time (division, size=503)
1e−7
successful search
unsuccessful search
Average Search Time (seconds)
Load Factor


Load Factor vs Search Time (division, size=1009)
1e−7
successful search
unsuccessful search
Average Search Time (seconds)
Load Factor

Key Distribution Comparison -

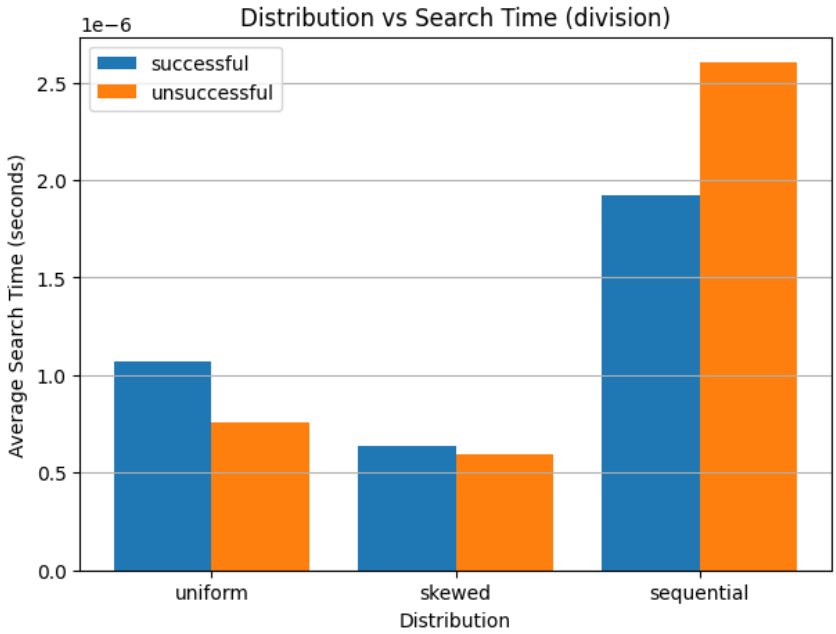
Another experiment compared the effect of different key distributions on search performance.

Three distributions were tested:

1. Uniform Distribution - Keys are randomly distributed across a large range.
2. Skewed Distribution - Most keys come from a small range, which increases the chance of collisions.
3. Sequential Keys - Keys are inserted in increasing order (1, 2, 3, …).

The experiments showed that:

- Uniform distribution generally gives the best performance because keys are spread evenly across the table.
- Skewed distributions often produce more collisions, which increases chain length and search time.
- Sequential keys can perform reasonably well if the hash function distributes them evenly.



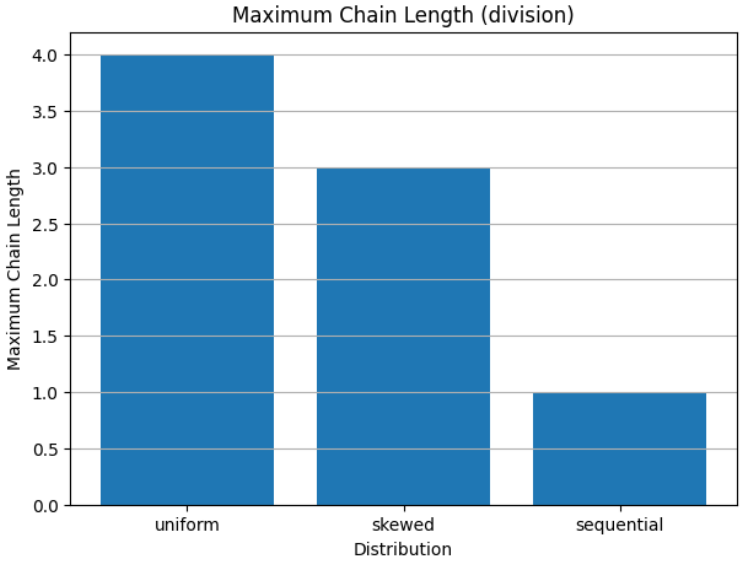
Memory Overhead Analysis

Since chaining was used for collision resolution, memory overhead was analyzed using:

- Average chain length per slot
- Maximum chain length

Average chain length indicates how many elements are stored in each slot on average, while maximum chain length represents the worst case.

The results showed that when the load factor increases or when keys are skewed, chains become longer. This increases memory overhead and slows down search operations.



3. Observations and Discussion

Effect of Load Factor

As the load factor increases, performance gradually degrades. When many elements are stored in the table relative to its size, collisions become more frequent and chains grow longer. This increases the time needed to search for keys.

In practice, keeping the load factor below a certain threshold (for example 0.75) can help maintain good performance.

Chaining vs Open Addressing

There are two common approaches for collision resolution: chaining and open addressing.

Chaining Advantages:

- Simple to implement
- Table can store more elements than its size
- Deletions are straightforward

Disadvantages:

- Uses additional memory for chains
- Cache performance may be worse because elements are scattered

Open Addressing Advantages:

- All elements are stored directly in the table
- Better memory locality

Disadvantages:

- Table cannot exceed its capacity
- Deletion is more complex
- Performance drops quickly as load factor approaches 1

Overall, chaining tends to handle high load factors better, while open addressing can be more memory-efficient when the load factor is kept low.

Effect of Key Distribution

Key distribution has a strong impact on hash table performance.

When keys are uniformly distributed, the hash function spreads them evenly across the table, resulting in short chains and fast searches.

However, skewed distributions cause many keys to map to the same indices, which increases collisions and chain lengths. This leads to slower search operations.

Sequential keys can perform well depending on the hash function. A good hash function should distribute sequential inputs evenly across the table.

Issues Encountered

One challenge during implementation was designing a hash function that works for multiple key types. To address this, keys were converted into numeric values before hashing.

Another limitation is the absence of dynamic resizing. Without resizing, the hash table may experience performance degradation when the load factor becomes very high.

4. Conclusion -

In this assignment, a hash table was implemented from scratch using Python lists and chaining for collision resolution. Two hashing methods (division and multiplication) were implemented and tested.

The experiments confirmed that search performance is strongly influenced by load factor and key distribution. Lower load factors generally lead to faster lookups, while skewed key distributions increase collisions and chain lengths.

Overall, the results demonstrate the importance of choosing appropriate hash functions and managing load factors to maintain efficient hash table performance.